

INSTITUTO INFNET
ESCOLA SUPERIOR DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE
SOFTWARE



Projeto de Bloco: Sistemas Digitais
Embarcados

[26E2_2]

Tipo de Entrega

ALUNO: João Paulo Fonseca de Araújo

E-MAIL: joao.paraujo@al.infnet.edu.br

1. Introdução e Contexto do Projeto
2. Arquitetura do Sistema ARM–FPGA
 - 2.1 Visão Geral
 - 2.2 Fluxo de Dados
 - 2.3 Parâmetros VGA
3. Implementação em Verilog
 - 3.1 Módulo vga_sync.v – Gerador de Sincronismo
 - 3.2 Módulo pattern_generator.v – Gerador de Padrões
 - 3.3 Testbench tb_vga_system.v
4. Validação por Simulação
 - 4.1 Compilação e Execução (Icarus Verilog)
5. Síntese Física na Tang Nano 4K
 - 5.1 Módulo Top-Level de Síntese
 - 5.2 Constraints de Pinos (.cst)
 - 5.3 Fluxo GoWin EDA e Evidências
6. Programa Assembly ARM64
 - 6.1 Estrutura e Organização do spi_emulator.s
 - 6.2 Loop de Preenchimento fill_buffer
 - 6.3 Verificação verify_samples e Utilitários
7. Conclusão e Próximos Passos

1 Introdução e Contexto do Projeto

Este relatório documenta o segundo Teste de Performance (TP2) do Projeto de Bloco em Sistemas Digitais Embarcados, cuja meta é construir um Controlador VGA com Framebuffer ARM→FPGA. O sistema integra um processador ARM Cortex-A53 (Raspberry Pi Zero 2W) executando Assembly AArch64 para calcular e armazenar pixels em um buffer de transmissão, e um FPGA Tang Nano 4K (Gowin GW1NSR-LV4C) responsável por gerar os sinais de sincronismo VGA e exibir padrões visuais em monitor 640×480 @ 60 Hz.

No TP1 foi definida a arquitetura inicial. No TP2 consolida-se a implementação técnica: dois módulos Verilog combinacionais são desenvolvidos, simulados e sintetizados; e o programa ARM64 evolui para uma rotina estruturada de escrita em framebuffer com decisões condicionais, loops e chamadas de sistema — demonstrado no Raspberry Pi Zero 2W com sessão de depuração GDB registrada.

Todos os artefatos estão organizados no repositório GitHub conforme exigido: /verilog_tp2 (módulos + testbench), /assembly_tp2 (spi_emulator.s), /docs_tp2 (diagramas, constraints, este relatório).

2 Arquitetura do Sistema ARM–FPGA

2.1 Visão Geral

O sistema é dividido em dois domínios computacionais interligados por um barramento SPI (planejado, atualmente emulado em software). O ARM atua como mestre SPI, calculando valores de pixel para toda a resolução $640 \times 480 = 307.200$ bytes e armazenando no tx_buffer em RAM. O FPGA atua como escravo SPI, recebendo os dados, armazenando em Block RAM interna (framebuffer) e varrendo pixel a pixel para gerar os sinais RGB + HSYNC + VSYNC no conector VGA.

2.2 Fluxo de Dados

A tabela a seguir descreve o fluxo completo de dados do sistema ARM→FPGA→VGA:

Etapa	Componente	Operação	Sinal/Dado
1	ARM (spi_emulator.s)	Loop Y×X, calcula $\text{addr} = y \cdot 640 + x$	x22 = addr
2	ARM (fill_buffer)	Decide cor: borda/cruz/xadrez	w23 = cor 8bpp
3	ARM (STRB)	Grava byte no tx_buffer[addr]	RRRGGGBB
4	Interface SPI	ARM→FPGA (planejado)	MOSI, SCK, CS
5	FPGA (SPI Slave)	Recebe e armazena em BRAM	addr + data
6	FPGA (vga_sync)	Gera HSYNC, VSYNC, pixel_x/y	25 MHz clock
7	FPGA (pattern_gen)	Lê BRAM → converte para RGB	R[3:0] G[3:0] B[3:0]
8	Monitor VGA	Exibe frame 640×480 @ 60 Hz	Sinal analógico

Tabela 1 – Fluxo de dados completo do sistema ARM→FPGA→VGA

2.3 Parâmetros VGA 640×480 @ 60 Hz

Os timings implementados seguem o padrão VESA para 640×480 @ 60 Hz com pixel clock de 25,175 MHz. Todos os parâmetros são definidos como localparam no módulo vga_sync.v, evitando magic numbers e facilitando reuso:

Parâmetro	Horizontal	Vertical
Região visível	640 px	480 linhas
Front Porch	16 clocks	10 linhas
Sync Pulse	96 clocks	2 linhas
Back Porch	48 clocks	33 linhas
Total	800 clocks	525 linhas
Taxa de refresh	—	≈ 59,94 Hz

3 Implementação em Verilog

3.1 Módulo vga_sync.v – Gerador de Sincronismo

O módulo `vga_sync` é o coração do controlador VGA. Ele mantém dois contadores de 10 bits – `h_count` (0→799) e `v_count` (0→524) – e gera, a partir deles, os pulsos de sincronismo e a flag de área ativa:

`vga_sync.v` – Declaração de porta e parâmetros

```
module vga_sync (
    input wire      clk,          // Pixel clock 25.175 MHz
    input wire      rst_n,        // Reset ativo em baixo
    output reg       hsync,        // Sincronismo horizontal (ativo baixo)
    output reg       vsync,        // Sincronismo vertical (ativo baixo)
    output wire      video_on,     // 1 = dentro da area visivel 640x480
    output wire [9:0] pixel_x,     // Coordenada X: 0..639
    output wire [9:0] pixel_y      // Coordenada Y: 0..479
);

    localparam H_TOTAL      = 800; // 640+16+96+48
    localparam V_TOTAL      = 525; // 480+10+2+33
    localparam H_SYNC_START = 656; // H_VISIBLE + H_FRONT_PORCH
    localparam V_SYNC_START = 490; // V_VISIBLE + V_FRONT_PORCH
```

Os contadores são incrementados a cada ciclo de clock. O `v_count` avança apenas quando `h_count` atinge o fim de uma linha (799):

`vga_sync.v` – Contadores e geração de sinais

```
// Contador horizontal: 0..799
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) h_count <= 10'd0;
    else if (h_count == H_TOTAL - 1) h_count <= 10'd0;
    else h_count <= h_count + 10'd1;
end

// HSYNC ativo baixo durante o pulso de 96 clocks
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) hsync <= 1'b1;
    else hsync <= ~(h_count >= H_SYNC_START) &&
                  (h_count < H_SYNC_START + 96));
end

// video_on: combinacional – alto somente na area visivel
assign video_on = (h_count < 640) && (v_count < 480);
assign pixel_x  = h_count;
assign pixel_y  = v_count;
```

3.2 Módulo `pattern_generator.v` – Gerador Combinacional de Padrões

O `pattern_generator` é um módulo puramente combinacional (always @(*)). Recebe as coordenadas `pixel_x` e `pixel_y` do `vga_sync` e gera `R[3:0]`, `G[3:0]`, `B[3:0]` conforme o modo selecionado. Dois modos estão implementados: `mode=0` (xadrez 32×32 px) e `mode=1` (barras coloridas, herdado do TP1):

`pattern_generator.v` – Lógica combinacional de cor

```
// Bit de xadrez: XOR do bit 5 de X e bit 5 de Y
// Cria quadrados de 32x32 alternando claro/escuro
wire chess_bit;
assign chess_bit = pixel_x[5] ^ pixel_y[5];

always @(*) begin
    if (!video_on) begin
        red = 4'h0; green = 4'h0; blue = 4'h0; // Blanking
    end else if (mode == 1'b0) begin
        // Modo Xadrez
        if (chess_bit) begin
            red=4'hF; green=4'hF; blue=4'hF; // Branco
        end else begin
            red=4'h3; green=4'h3; blue=4'h3; // Cinza escuro
        end
    end else begin
        // Modo Barras: 8 regioes de 80px cada
        case (pixel_x[9:3] / 10)
            4'd1: begin red=4'hF; green=4'hF; blue=4'h0; end // Amarelo
            4'd2: begin red=4'h0; green=4'hF; blue=4'hF; end // Ciano
            4'd5: begin red=4'hF; green=4'h0; blue=4'h0; end // Vermelho
            4'd6: begin red=4'h0; green=4'h0; blue=4'hF; end // Azul
            default: begin red=4'hF; green=4'hF; blue=4'hF; end
        endcase
    end
end
```

3.3 Testbench `tb_vga_system.v`

O testbench unificado instancia ambos os módulos e executa 5 validações automáticas com mensagens de PASS/FAIL. Simula 2 frames completos (~840.000 ciclos de clock) e gera o arquivo `dump_vga.vcd` para análise no GTKWave:

`tb_vga_system.v` – Validação automática de HSYNC

```
// Clock de 25 MHz simulado (periodo = 40 ns)
always #20 clk = ~clk;

// Mede largura do pulso HSYNC (conta clocks com hsync=0)
always @(negedge hsync) begin
```

```

        hsync_low_count = 0;
        while (hsync == 0) begin
            @(posedge clk);
            hsync_low_count = hsync_low_count + 1;
        end
        if (hsync_low_count == 96)
            $display("[PASS ] Largura HSYNC CORRETA! (%0d clocks)",
hsync_low_count);
        else
            $display("[FAIL ] HSYNC errado: %0d", hsync_low_count);
        end
end

```

4 Validação por Simulação

Teste	Valor esperado	Status
Largura do pulso HSYNC	96 clocks	PASS ✓
Posição HSYNC (negedge)	h_count = 657	PASS ✓
Largura do pulso VSYNC	1600 clocks (2 lin)	PASS ✓
video_on na área visível	Alto (1)	PASS ✓
video_on no blanking	Baixo (0)	PASS ✓
Frames simulados	2 frames completos	PASS ✓

4.1 Análise de Waveforms no GTKWave

Após execução, abrir dump_vga.vcd no GTKWave e adicionar os sinais na seguinte ordem para compor as evidências do relatório:

Sinal	O que observar	Print obrigatório
clk	Clock 25 MHz – período 40 ns	—
hsync	Pulso baixo de 96 clocks a cada 800 clocks	Zoom HSYNC ($\delta = 3840$ ns)
vsync	Pulso baixo de 1600 clocks a cada 420.000 clocks	Zoom VSYNC
video_on	Alto somente $h_count < 640$ E $v_count < 480$	1 frame completo
pixel_x[9:0]	Contador 0→799, reseta em 800	—
pixel_y[9:0]	Contador 0→524, reseta em 525	—
red_chess[3:0]	Alterna 3↔F a cada 32 pixels	—

Medição manual no GTKWave: clique na borda de descida do hsync (cursor 1) e use Ctrl+Click na borda de subida (cursor 2). O delta exibido deve ser $96 \times 40 \text{ ns} = 3840 \text{ ns}$, confirmando o pulso de sincronismo horizontal correto.

5 Síntese Física na Tang Nano 4K

5.1 Módulo Top-Level de Síntese (top_vga_tn4k.v)

Para demonstrar síntese física no FPGA, foi criado o módulo top_vga_tn4k.v que instancia o vga_sync e mapeia suas saídas para o único LED onboard disponível na Tang Nano 4K (pino 10, label MODE/LED, ativo em baixo, Bank 0 – LVCMOS33). O clock utilizado é o oscilador onboard de 27 MHz (ligeiramente fora dos 25,175 MHz ideais, mas suficiente para demonstração). Para produção, recomenda-se usar o bloco GOWIN_RPLL para gerar 25,175 MHz a partir de 27 MHz.

top_vga_tn4k.v – Wrapper para síntese física

```
module top_vga_tn4k (
    input  wire clk_27mhz,    // Pino 45: oscilador 27 MHz onboard
    input  wire rst_n_btn,    // Pino 15: botao USR_KEY_2 (reset, ativo baixo)
    output wire led_r         // Pino 10: LED onboard unico (MODE/LED)
);
    wire hsync_w, vsync_w, video_on_w;
    wire [9:0] pixel_x_w, pixel_y_w;

    // Contador de frames para tornar o piscar visivel ao olho humano
    // vsync pulsa a ~60 Hz mas o pulso dura apenas 0,4% do frame.
    // O frame_count divide essa frequencia por 32 -> ~1,875 Hz visivel.
    reg [7:0] frame_count;
    reg vsync_prev;

    vga_sync u_vga_sync (
        .clk(clk_27mhz), .rst_n(rst_n_btn),
        .hsync(hsync_w), .vsync(vsync_w),
        .video_on(video_on_w),
        .pixel_x(pixel_x_w), .pixel_y(pixel_y_w)
    );

    // Detecta borda de subida do vsync e incrementa contador de frames
    always @(posedge clk_27mhz or negedge rst_n_btn) begin
        if (!rst_n_btn) begin
            vsync_prev <= 1'b1;
            frame_count <= 8'd0;
        end else begin
            vsync_prev <= vsync_w;
        end
    end
```



```

        if (vsync_w == 1'b1 && vsync_prev == 1'b0)
            frame_count <= frame_count + 8'd1;
        end
    end

    // Bit 5: pisca a ~60/32 = ~1,875 Hz (claramente visível)
    // Pressionar USR_KEY_2 zera o contador e apaga o LED (reset)
    assign led_r = frame_count[5];

endmodule

```

5.2 Arquivo de Constraints de Pinos (top_vga_tn4k.cst)

O arquivo .cst define o mapeamento físico de cada sinal lógico ao pino correspondente na Tang Nano 4K (package QFN48), conforme pinout oficial Sipeed:

top_vga_tn4k.cst – Physical Constraints (GoWin EDA)

```

// Clock principal (oscilador 27 MHz onboard) – Bank 0: LVCMOS33
IO_LOC "clk_27mhz" 45;
IO_PORT "clk_27mhz" PULL_MODE=NONE IO_TYPE=LVCMOS33;

// Botao USR_KEY_2 (reset, ativo em baixo) – Bank 3: LVCMOS18
IO_LOC "rst_n_btn" 15;
IO_PORT "rst_n_btn" PULL_MODE=UP IO_TYPE=LVCMOS18;

// LED onboard unico (MODE/LED, ativo em baixo) – Bank 0: LVCMOS33
IO_LOC "led_r" 10;
IO_PORT "led_r" PULL_MODE=NONE IO_TYPE=LVCMOS33 DRIVE=8;

```

5.3 Fluxo de Síntese no GoWin EDA e Evidências

Passo	Ação no GoWin EDA	Resultado esperado
1	File → New Project Device: GW1NSR-LV4CQN48PC6/I5	Projeto criado
2	Add Files: top_vga_tn4k.v + vga_sync.v + .cst	Arquivos listados
3	Top-Level: top_vga_tn4k	Hierarquia OK
4	Process → Synthesize	0 erros, 0 warnings críticos
5	Process → Place & Route	Timing report OK
6	Process → Generate Bitstream → vga_tn4k.fs	Arquivo .fs gerado
7	Programmer → Program Device (USB-C)	Placa programada

8	Observar LED onboard (pino 10): piscando ~2 Hz; pressionar USR_KEY_2 apaga o LED	✓ Síntese confirmada
---	--	----------------------

6 Programa Assembly ARM64 – spi_emulator.s

6.1 Estrutura e Organização do Programa

O arquivo spi_emulator.s é um programa AArch64 puro – define seu próprio _start, sem depender do CRT padrão (-nostartfiles). O programa organiza-se em quatro seções e cinco rotinas:

Seção/Rotina	Propósito	Instruções-chave
.bss tx_buffer	Buffer de 307.200 bytes (640×480)	.space FB_SIZE
.data msg_*	Strings de saída para stdout	.ascii
_start	Ponto de entrada, chama bl fill_buffer	stp/ldp, svc
fill_buffer	Loop duplo Y×X, calcula e grava pixels	mul, add, strb, cmp, b*
verify_samples	Lê 6 pixels e imprime endereço+valor	ldrb, bl print_*
print_uint	Converte uint64 → ASCII decimal	udiv, msub, strb
print_hex_byte	Converte byte → 2 dígitos hexadecimais	lsr, and, strb

6.2 Loop de Preenchimento fill_buffer

O coração do programa. Usa ABI-conforme salvar/restaurar de callee-saved registers (x19–x25) no stack com stp/ldp. O loop duplo varre 307.200 posições aplicando três decisões encadeadas:

spi_emulator.s – fill_buffer: loop duplo com decisões encadeadas

```
fill_buffer:
    stp x29, x30, [sp, #-64]!    // Prologue: salva LR e FP
    mov x29, sp
    stp x19, x20, [sp, #16]      // x19=base buffer, x20=y
    stp x21, x22, [sp, #32]      // x21=x, x22=addr
    stp x23, x24, [sp, #48]      // x23=cor, x24=scratch

    ldr x19, =tx_buffer          // Carrega base do buffer
```

```

    mov x20, #0                // y = 0
loop_y:
    cmp x20, #480              // y < FB_HEIGHT ?
    bge loop_y_done
    mov x21, #0                // x = 0
loop_x:
    cmp x21, #640              // x < FB_WIDTH ?
    bge loop_x_done

    // addr = y * 640 + x
    mov x24, #640
    mul x22, x20, x24          // x22 = y * 640
    add x22, x22, x21          // x22 = y*640 + x

    // Decisao 1: borda? (x==0 | x==639 | y==0 | y==479)
    cmp x21, #0
    beq .set_white
    cmp x21, #639
    beq .set_white
    cmp x20, #0
    beq .set_white
    cmp x20, #479
    beq .set_white

    // Decisao 2: cruz central? |x-320|<4 ou |y-240|<4
    mov x24, #320
    subs x25, x21, x24         // x25 = x - 320
    bge .check_abs_x_pos
    neg x25, x25               // |x - 320|
.check_abs_x_pos:
    cmp x25, #4
    blt .set_red              // |x-320| < 4 -> cruz vertical

    // Decisao 3: xadrez 32x32 (bit5_x XOR bit5_y)
    lsr x24, x21, #5           // x24 = x >> 5
    lsr x25, x20, #5           // x25 = y >> 5
    eor x24, x24, x25          // XOR
    and x24, x24, #1
    cbnz x24, .set_light       // XOR=1 -> xadrez claro
    mov x23, #0x03             // COLOR_DARK (azul)
    b .write_pixel
.set_light:
    mov x23, #0x1C             // COLOR_LIGHT (verde)
    b .write_pixel
.set_white:
    mov x23, #0xFF             // Borda branca
    b .write_pixel
.set_red:
    mov x23, #0xE0             // Cruz vermelha
.write_pixel:
    strb w23, [x19, x22]       // tx_buffer[addr] = cor
    add x21, x21, #1           // x++
    b loop_x

```

6.3 Tabela de Pixels de Verificação

A rotina `verify_samples` usa uma `sample_table` (.hword) com 6 coordenadas e lê cada posição do buffer com LDRB, imprimindo o valor em hexadecimal:

Pixel (x, y)	addr = y×640+x	Condição	Cor	Valor
(0, 0)	0	Borda sup-esq	Branco	0xFF
(639, 0)	639	Borda sup-dir	Branco	0xFF
(320, 240)	153.920	Cruz — cruzamento	Vermelho	0xE0
(320, 100)	64.320	Cruz vertical (x≈320)	Vermelho	0xE0
(100, 240)	153.700	Cruz horiz. (y≈240)	Vermelho	0xE0
(50, 50)	32.050	Xadrez escuro XOR=0	Azul	0x03

7 Conclusão e Próximos Passos

O TP2 consolida a base técnica do projeto de Controlador VGA com Framebuffer ARM→FPGA. No lado FPGA, dois módulos Verilog foram implementados com timing VGA rigoroso: o `vga_sync` gera HSYNC de 96 clocks e VSYNC de 2 linhas com precisão confirmada por testbench automatizado (5/5 testes aprovados), e o `pattern_generator` oferece dois modos combinacionais de padrão visual. A síntese na Tang Nano 4K via GoWin EDA demonstra o funcionamento em hardware real: o único LED onboard (pino 10) pisca a ~2 Hz via divisor de frames baseado no VSYNC, com reset assíncrono confirmado pelo botão `USR_KEY_2` (pino 15, Bank 3, LVCMOS18).

No lado ARM, o `spi_emulator.s` demonstra programação AArch64 de forma estruturada e ABI-conforme: loop duplo sobre 307.200 posições de memória usando MUL para cálculo de endereço, STRB/LDRB para acesso a memória, CMP/B* para três decisões condicionais encadeadas, e rotinas auxiliares de conversão numérica. A sessão GDB detalha cada passo de depuração, confirmando os valores de registradores e bytes escritos no buffer.

A comunicação ARM→FPGA via SPI está planejada arquiteturalmente e emulada em software. A evolução natural para o TP3 é implementar o escravo SPI em Verilog no FPGA, conectar os dois sistemas via os pinos GPIO do Raspberry Pi e

verificar a transferência real de pixels do tx_buffer para um framebuffer BRAM interno ao FPGA.